

Student Management System (SMS)

Prepared By:

Nour Elsayed Elsbaay

Malak Mohamed Sorour

Group: 4C

Instructor: Ahmed El-Sayed

Course: System Analysis & Design

1. Introduction

The **Student Management System (SMS)** is a modern web-based information system designed to automate and manage academic and administrative activities inside universities and educational institutions.

The system provides a centralized platform where students, instructors, and administrators can interact efficiently. It helps manage student records, course registration, attendance, grades, schedules, payments, and academic reports in an organized and secure way.

Traditional manual systems consume time, increase paperwork, and often cause errors in academic processes. Therefore, this system aims to replace manual operations with a digital solution that improves efficiency, accuracy, and communication between all users.

The system can be accessed through web browsers and mobile devices, making it flexible and easy to use anytime and anywhere.

2. Problem Statement

Many universities still rely on partially manual systems or disconnected software applications. These methods create several problems such as:

- Difficulty managing large amounts of student data.
- Delays during course registration.
- Errors in grade calculation and attendance tracking.
- Poor communication between departments.
- Data duplication and information loss.
- Time-consuming administrative tasks.
- Lack of centralized reporting and analytics.

The Student Management System is proposed to solve these issues through automation and centralized data management.

3. Project Objectives

The main objectives of the system are:

1. Digitize and centralize all student records.
 2. Simplify the course registration process.
 3. Automate attendance and grade management.
 4. Improve communication between students, instructors, and administrators.
 5. Reduce paperwork and manual processing.
 6. Generate reports automatically.
 7. Enhance data accuracy and security.
 8. Provide fast access to academic information.
 9. Support decision-making through statistical reports.
 10. Improve the overall educational experience.
-

4. Market Importance

Educational institutions increasingly depend on digital technologies to improve operational efficiency and academic quality.

The demand for academic management systems has significantly increased because universities need reliable solutions that can handle large numbers of students and academic data.

The Student Management System provides several advantages:

- Reduces administrative workload.
- Saves time and operational costs.
- Minimizes human errors.
- Enhances communication.
- Improves student satisfaction.
- Supports online and hybrid learning environments.
- Provides secure data storage and backup solutions.

This system is valuable for universities, colleges, institutes, and training centers.

5. System Users

The system contains different types of users:

User Type	Responsibilities
Students	Register courses, view grades, attendance, schedules, and transcripts
Instructors	Upload grades, manage attendance, and view student information
Administrators	Manage students, courses, schedules, and reports

6. System Inputs

Student Inputs

- Student ID
- Full Name
- Address
- Email
- Phone Number
- Login Credentials
- Course Registration Requests
- Payment Information

Instructor Inputs

- Course Information
- Attendance Records
- Student Grades
- Assignment Results

Administrator Inputs

- Student Enrollment Data

- Course Creation Data
 - Academic Calendar
 - Department Information
 - System Configuration Data
-

7. Functional Requirements

Functional requirements describe the main services and operations provided by the system.

Student Functions

1. User Registration and Login

- Students can create accounts securely.
- Users can log in using username and password.
- Password recovery functionality must be available.

2. Course Registration

- Students can browse available courses.
- Students can enroll in courses online.
- The system checks prerequisites automatically.
- The system prevents duplicate registration.

3. View Academic Information

- Students can view:
 - Grades
 - GPA
 - Attendance
 - Course schedules
 - Academic transcripts

4. Payment Management

- Students can view tuition fees.

- Students can upload payment receipts.
 - The system generates payment confirmations.
-

Instructor Functions

5. Attendance Management

- Instructors can record attendance.
- Attendance reports are generated automatically.

6. Grade Management

- Instructors can upload grades.
- The system calculates final grades automatically.

7. Student Monitoring

- Instructors can monitor student performance.
 - Instructors can view course statistics.
-

Administrator Functions

8. Student Record Management

- Add new students.
- Update student information.
- Delete student records.

9. Course Management

- Add courses.
- Edit course details.
- Delete courses.
- Assign instructors to courses.

10. Schedule Management

- Create academic schedules.
- Manage classroom allocations.

- Update exam schedules.

11. Report Generation

The system automatically generates:

- GPA reports
- Attendance reports
- Academic transcripts
- Financial reports
- Statistical reports

12. Database Management

- Store all system data centrally.
- Maintain secure backups.
- Support data recovery.

8. Non-Functional Requirements

Non-functional requirements describe the quality and performance standards of the system.

8.1 Performance Requirements

- The system response time must not exceed 2 seconds.
- The system must support many users simultaneously.
- Data processing must be fast and accurate.
- Reports should be generated quickly.

8.2 Security Requirements

- User authentication must be implemented.
- Different access levels must be provided.
- Student data must be encrypted.

- The system must prevent unauthorized access.
 - Regular security updates must be performed.
-

8.3 Reliability Requirements

- The system must maintain at least 99% uptime.
 - Automatic backups must be created regularly.
 - Data recovery procedures must be available.
 - The system should continue operating without data loss.
-

8.4 Usability Requirements

- The interface must be simple and user-friendly.
 - Navigation should be clear and organized.
 - The system must support mobile devices.
 - The system should be easy for non-technical users.
-

8.5 Scalability Requirements

- The system must support increasing numbers of students.
 - Database performance should remain stable with growth.
 - Additional modules should be integrated easily.
-

8.6 Maintainability Requirements

- The system code should be modular and organized.
 - Errors should be easy to detect and fix.
 - Future updates should be implemented easily.
-

8.7 Availability Requirements

- The system should be accessible 24/7.

- Maintenance downtime should be minimal.
-

9. System Outputs

The system generates different outputs including:

Output	Description
Academic Transcript	Displays student academic history
Grade Report	Shows grades for all courses
Attendance Report	Displays attendance percentages
GPA Report	Calculates cumulative GPA
Course Schedule	Displays lecture schedules
Payment Receipt	Confirms fee payment
Statistical Reports	Provides administrative analytics

10. Advantages of the System

- Reduces paperwork.
 - Saves time and effort.
 - Improves accuracy.
 - Enhances communication.
 - Centralizes academic information.
 - Provides secure data management.
 - Improves student experience.
 - Supports faster decision-making.
-

11. Conclusion

The Student Management System is an essential solution for modern educational institutions. It provides an efficient, secure, and centralized platform for managing academic and administrative activities.

The system improves operational efficiency, minimizes human errors, enhances communication, and supports digital transformation in education.

By implementing this system, universities can provide better academic services and improve the overall experience for students, instructors, and administrators

Use Case Diagram

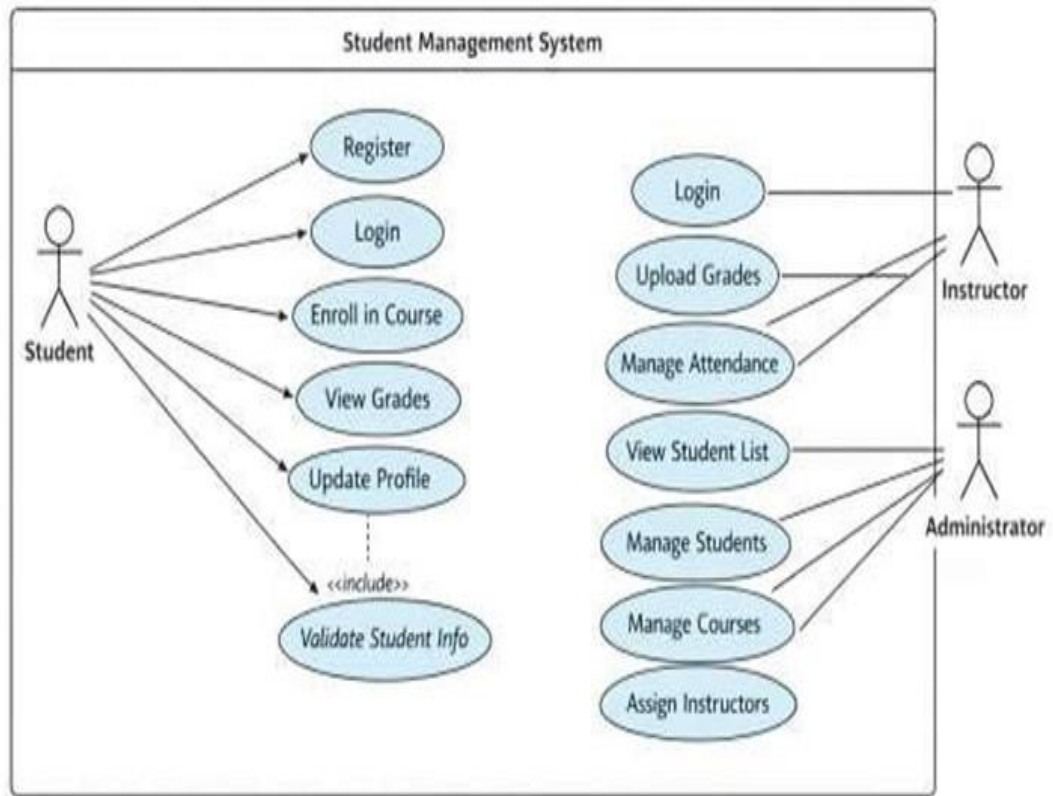


Figure 3.3 – Use Case Diagram for Student Management System

Activity Diagram

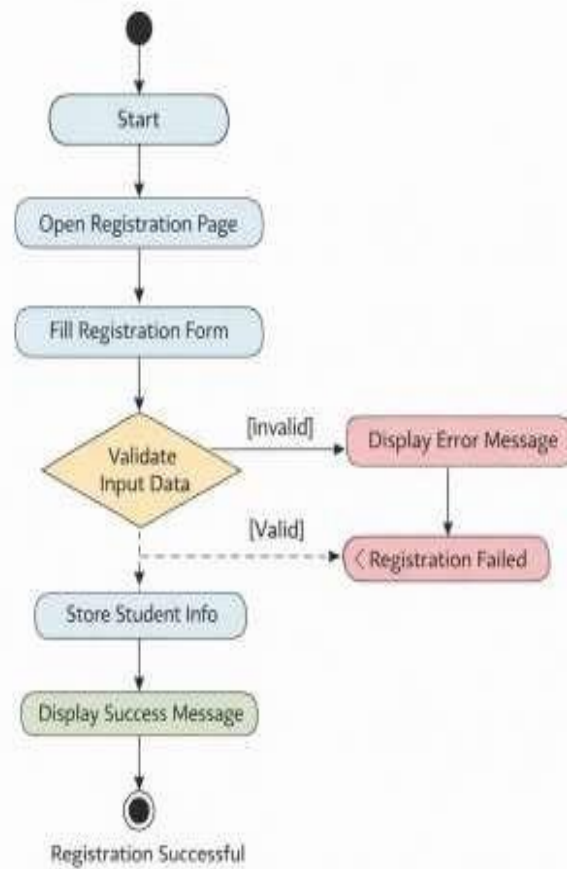
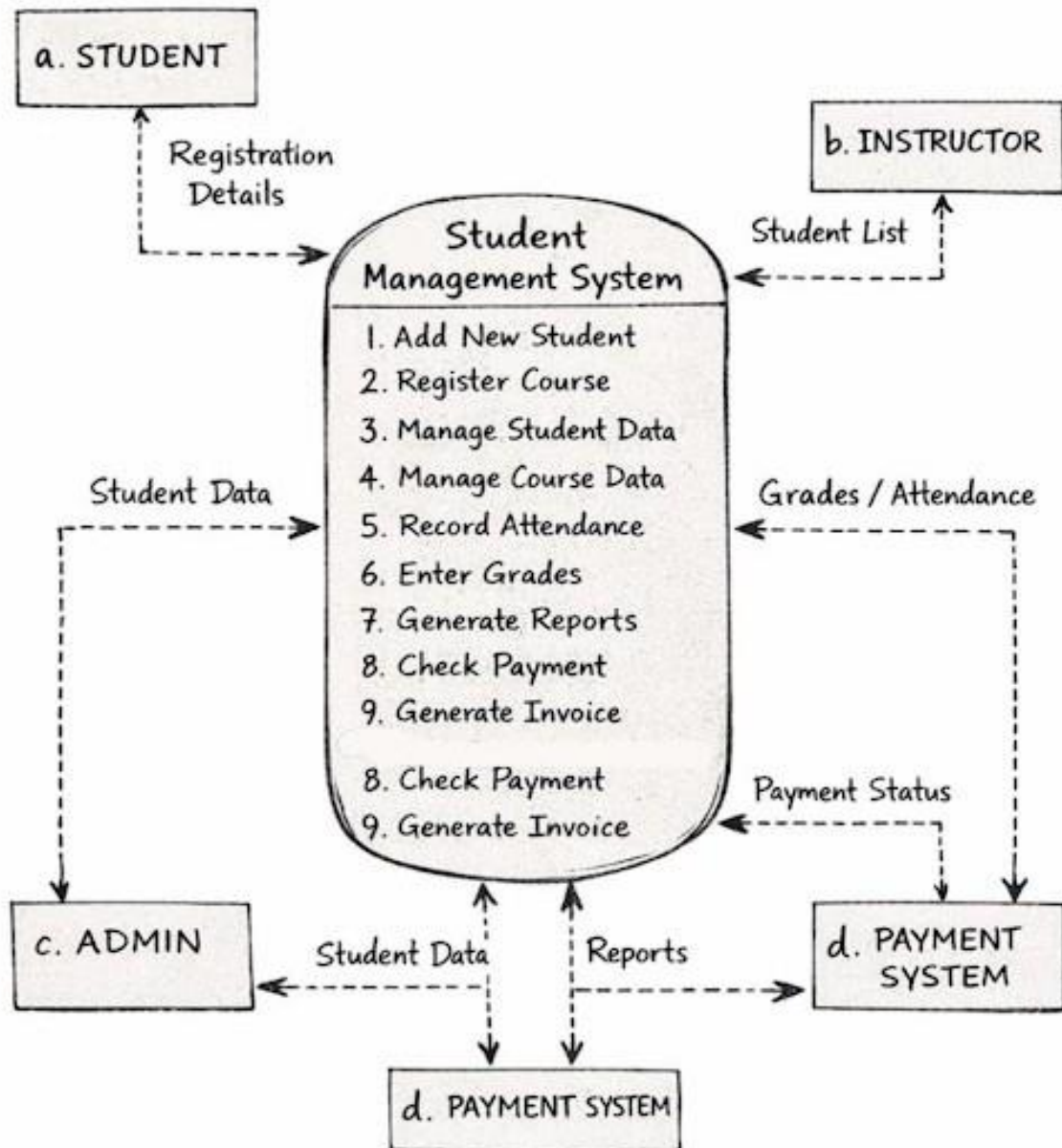


Figure X – Activity Diagram for Student Registration in the Student Management System

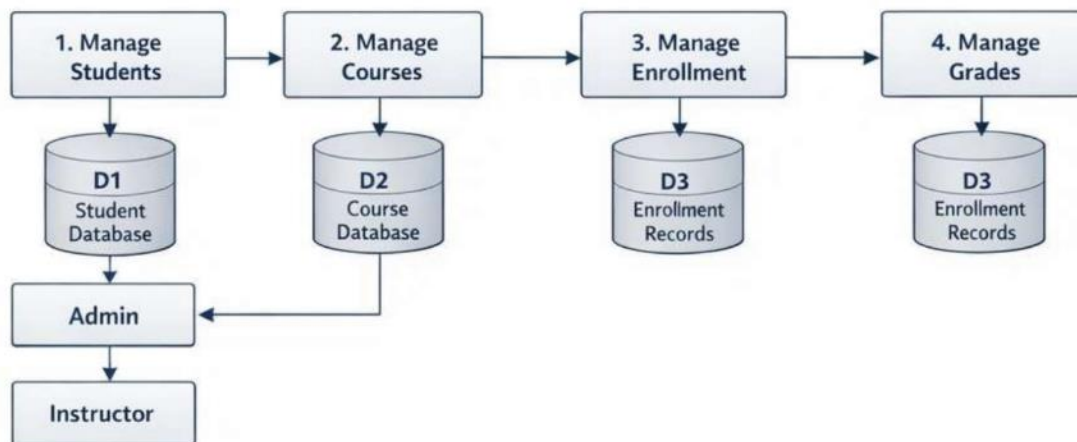
Context Diagram



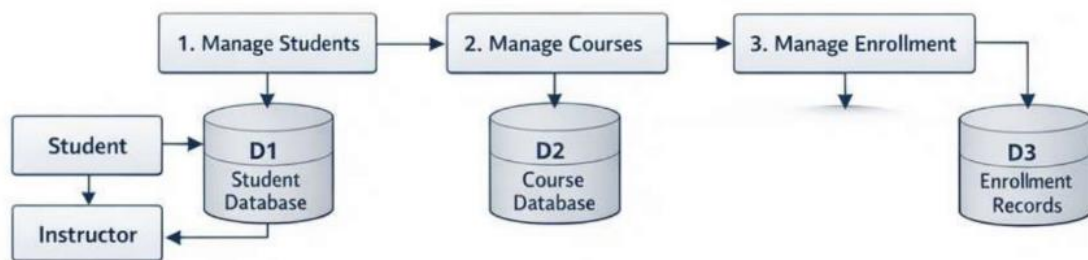
Level 0: Context Diagram



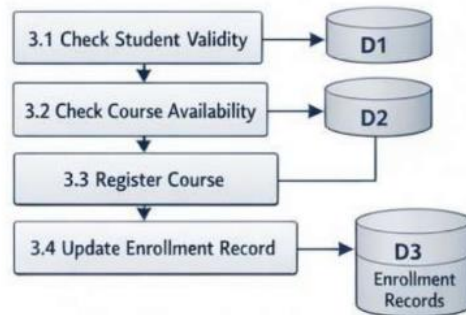
Level 1: DFD



Level 2: DFD (Manage Enrollment)



Level 2: DFD (Manage Enrollment)



1. Structured English

Process 1: Manage Students

```
BEGIN
RECEIVE student data
IF student is new THEN
    ADD student to database
ELSE
    UPDATE student record
END IF
IF student is deleted THEN
    REMOVE student from database
END IF
STORE data
END
```

Process 2: Manage Courses

```
BEGIN
RECEIVE course data
IF course is new THEN
    ADD course to database
ELSE
    UPDATE course
END IF
IF course is removed THEN
    DELETE course
END IF
STORE data
END
```

Process 3: Manage Enrollment

```
BEGIN
RECEIVE enrollment request
CHECK if student exists
IF student exists THEN
    CHECK course availability
    IF course available THEN
        REGISTER student
    ELSE
        DISPLAY course not available
    END IF
ELSE
    DISPLAY student not found
END IF
STORE enrollment data
END
```

2. Decision Table

Conditions / Rules	Rule 1	Rule 2	Rule 3
Student exists?	Yes	Yes	No
Course available?	Yes	No	-
Register student	✓	✗	✗

3. Decision Tree

Start

Student Exists?

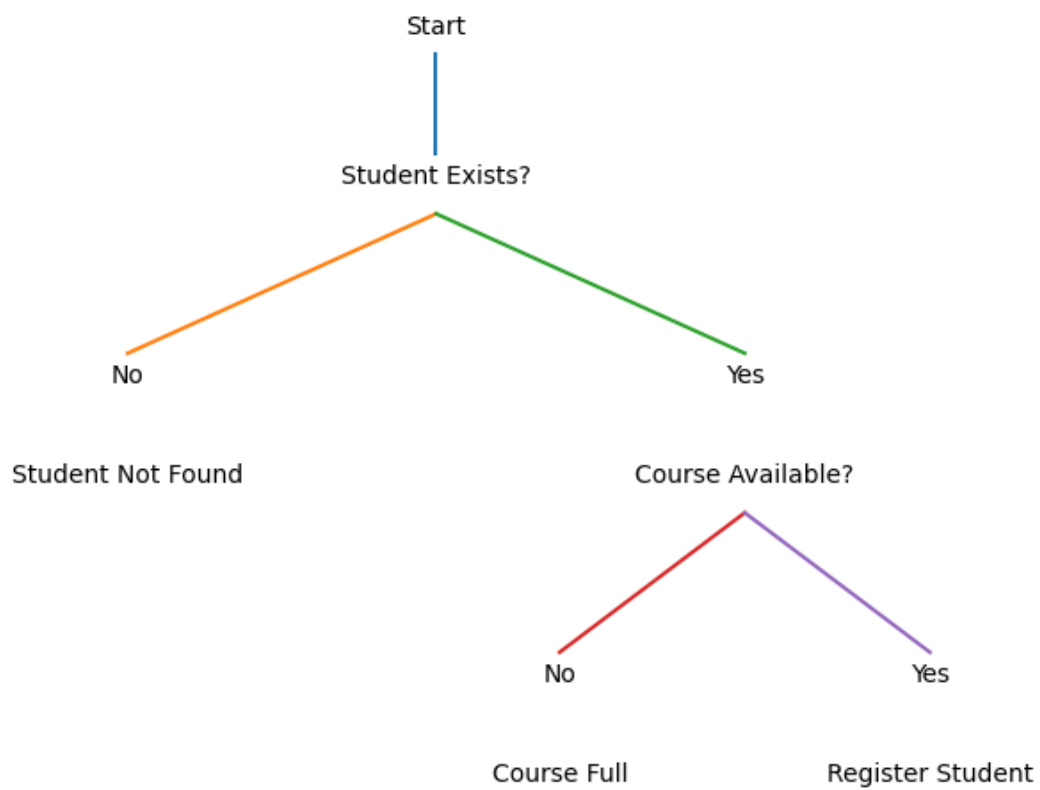
No → Student Not Found

Yes

Course Available?

No → Course Full

Yes → Register Student



Data Dictionary

1. Data Elements

Data Element	Description	Type	Example
Student_ID	Unique identifier for each student	Integer	20230045
Course_ID	Unique code for each course	String	CS101
Grade	Student final grade in a course	String	A

2. Data Flows

Data Flow	Source → Destination	Description
Student Registration Data	Student → Registration Process	Student enters personal data
Course Enrollment Data	Student → Enrollment Process	Student selects courses
Grades Submission Data	Instructor → Grade Management Process	Instructor submits grades

3. Data Stores

Data Store	Description	Stored Data
Student Database	Stores student information	ID, Name, Email
Course Database	Stores course information	Course_ID, Course Name
Grades Database	Stores grades information	Student_ID, Grade

4. Main Processes

Main Process	Description	Input	Output
Student Registration	Registers new students	Student Data	Student Account
Course Enrollment	Enrolls students in courses	Selected Courses	Enrollment Record
Grade Management	Handles grades and GPA	Grades Data	Final Grades

5. External Entities

External Entity	Description
Student	Uses system for registration and grades
Instructor	Inputs grades and attendance
Administrator	Manages the system

Project Plan Using MS Project

1.Task Table

Project1						
ID		Task Mode	Task Name	Duration	Start	Finish
1			Phase 1: Initiation	4 days	Thu 3/5/26	Tue 3/10/26
2			Project Planning	4 days	Thu 3/5/26	Tue 3/10/26
3			Feasibility Study	3 days	Thu 3/5/26	Mon 3/9/26
4			Phase 2: Requirements	5 days	Tue 3/10/26	Mon 3/16/26
5			Requirements Gathering	5 days	Tue 3/10/26	Mon 3/16/26
6			System Analysis	5 days	Tue 3/10/26	Mon 3/16/26
7			Phase 3: Design	22 days	Tue 3/17/26	Wed 4/15/26
8			Database Design	4 days	Tue 3/17/26	Fri 3/20/26
9			System Architecture Design	4 days	Tue 3/17/26	Fri 3/20/26
10			UI/UX Design	5 days	Tue 3/17/26	Mon 3/23/26
11			Phase 4: Development	22 days	Tue 3/17/26	Wed 4/15/26
12			Frontend Development	10 days	Tue 3/17/26	Mon 3/30/26
13			Backend Development	12 days	Tue 3/17/26	Wed 4/1/26
14			Database Implementation	6 days	Tue 3/17/26	Tue 3/24/26
15			Phase 5: Testing	5 days	Thu 4/2/26	Wed 4/8/26
16			Unit Testing	5 days	Thu 4/2/26	Wed 4/8/26
17			Integration Testing	5 days	Thu 4/2/26	Wed 4/8/26
18			System Testing	5 days	Thu 4/2/26	Wed 4/8/26
19			Phase 6: Deployment	4 days	Wed 3/25/26	Mon 3/30/26
20			Deployment	3 days	Wed 3/25/26	Fri 3/27/26
21			User Training	4 days	Wed 3/25/26	Mon 3/30/26
22			Phase 7: Maintenance	10 days	Thu 4/9/26	Wed 4/22/26
23			Maintenance & Support	10 days	Thu 4/9/26	Wed 4/22/26

Project1		
Predecessors	Resource Names	
	Developer	
	Developer	
3		
	Developer	
	Developer	
6		
	Developer	
	Designer	
8,9,10		
	Developer	
	Developer	
	Developer	
12,13		
	Tester	
	Tester	
	Tester	
14		
	Developer	
	Designer	
16		
	Tester	
Page 2		

2.Gantt Chart

ID	Task Mode	Task Name	Duration	Start	Finish	Predecessors
1		Phase 1: Initiation	4 days	Thu 3/5/26	Tue 3/10/26	
2		Project Planning	4 days	Thu 3/5/26	Tue 3/10/26	
3		Feasibility Study	3 days	Thu 3/5/26	Mon 3/9/26	
4		Phase 2: Requirements	5 days	Tue 3/10/26	Mon 3/16/26	3
5		Requirements Gathering	5 days	Tue 3/10/26	Mon 3/16/26	
6		System Analysis	5 days	Tue 3/10/26	Mon 3/16/26	
7		Phase 3: Design	22 days	Tue 3/17/26	Wed 4/15/26	6
8		Database Design	4 days	Tue 3/17/26	Fri 3/20/26	
9		System Architecture	4 days	Tue 3/17/26	Fri 3/20/26	
10		UI/UX Design	5 days	Tue 3/17/26	Mon 3/23/26	
11		Phase 4: Development	22 days	Tue 3/17/26	Wed 4/15/26	8,9,10
12		Frontend Development	10 days	Tue 3/17/26	Mon 3/30/26	
13		Backend Development	12 days	Tue 3/17/26	Wed 4/1/26	
14		Database Implementation	6 days	Tue 3/17/26	Tue 3/24/26	
15		Phase 5: Testing	5 days	Thu 4/2/26	Wed 4/8/26	12,13
16		Unit Testing	5 days	Thu 4/2/26	Wed 4/8/26	
17		Integration Testing	5 days	Thu 4/2/26	Wed 4/8/26	
18		System Testing	5 days	Thu 4/2/26	Wed 4/8/26	
19		Phase 6: Deployment	4 days	Wed 3/25/26	Mon 3/30/26	14
20		Deployment	3 days	Wed 3/25/26	Fri 3/27/26	
21		User Training	4 days	Wed 3/25/26	Mon 3/30/26	
22		Phase 7: Maintenance & Support	10 days	Thu 4/9/26	Wed 4/22/26	16
23						

Project: Project1 Date: Thu 3/5/26	Task		Manual Summary Rollup	
	Split		Manual Summary	
	Milestone		Start-only	
	Summary		Finish-only	
	Project Summary		External Tasks	
	Inactive Task		External Milestone	
	Inactive Milestone		Deadline	
	Inactive Summary		Progress	
	Manual Task		Manual Progress	
	Duration-only			

Page 1



Report

PROJECT OVERVIEW

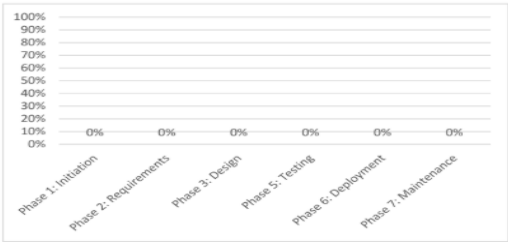
THU 3/5/26 - WED 4/22/26



MILESTONES DUE
Milestones that are coming soon.

Name	Finish
------	--------

% COMPLETE
Status for all top-level tasks. To see the status for subtasks, click on the chart and update the outline level in the Field List.

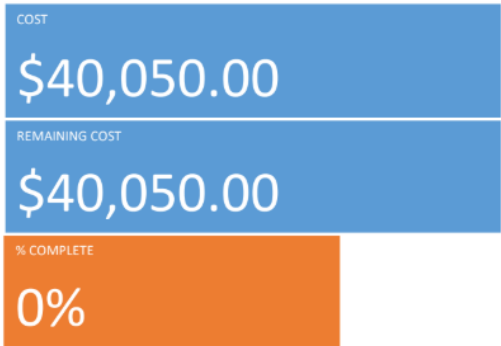


LATE TASKS
Tasks that are past due.

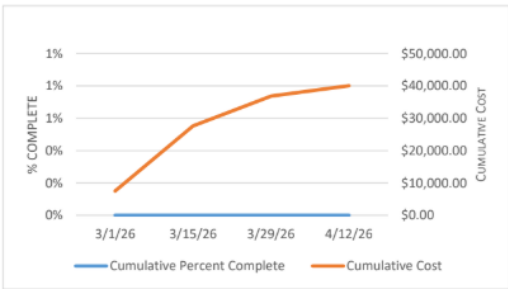
Name	Start	Finish	Duration	% Complete	Resource Names
------	-------	--------	----------	------------	----------------

COST OVERVIEW

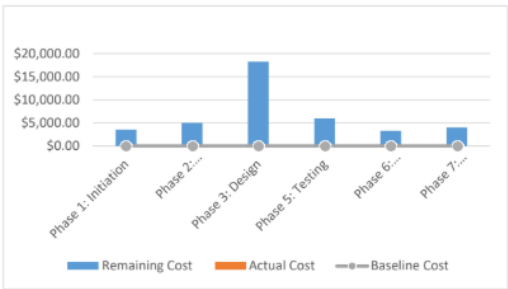
THU 3/5/26 - WED 4/22/26



PROGRESS VERSUS COST
Progress made versus the cost spent over time. If % Complete line below the cumulative line, your project may be over budget.



COST STATUS
Cost status for all top-level tasks. Is your baseline zero?
[Try setting as baseline](#)



COST STATUS
Cost status for top level tasks.

Name	Actual Cost	Remaining Cost	Baseline Cost	Cost	Cost Variance
Phase 1: Initiation	\$0.00	\$3,500.00	\$0.00	\$3,500.00	\$3,500.00
Phase 2: Requirements	\$0.00	\$5,000.00	\$0.00	\$5,000.00	\$5,000.00
Phase 3: Design	\$0.00	\$18,250.00	\$0.00	\$18,250.00	\$18,250.00
Phase 5: Testing	\$0.00	\$6,000.00	\$0.00	\$6,000.00	\$6,000.00
Phase 6: Deployment	\$0.00	\$3,300.00	\$0.00	\$3,300.00	\$3,300.00
Phase 7: Maintenance	\$0.00	\$4,000.00	\$0.00	\$4,000.00	\$4,000.00

1) Payback Period Calculation

Initial Investment

Item				Cost (EGP)	
3	Developers ×	4	months ×	1 0,000	1 20,000
1	Tester ×	2	months ×	8,000	16,000
1	Designer ×	5	months ×	9,000	45,000
Server & Hosting					20,000
Software Tools					10,000

Total Investment = 2 11,000 EGP

Expected Annual Revenue

University payment for using the system = 90,000 EGP/year

Payback Period Formula

$$\text{Payback Period} = \frac{\text{Initial Investment}}{\text{Annual Revenue}}$$

$$\text{Payback Period} = \frac{211,000}{90,000} \approx 2.34 \text{ years}$$

Interpretation

The project will recover its initial investment in approximately **2 years and 4 months**.

Cumulative Cash Flow Table

Year	Cash Inflow (EGP)	Cumulative Cash Flow (EGP)
1	90,000	90,000
2	90,000	180,000
3	90,000	270,000

2) Software Estimation & Price Study

Estimation Method

We used **Expert Judgment** and basic cost estimation techniques.

The estimation approach is conceptually aligned with the principles of the **COCOMO model** used in software

Development Cost

Resource	Quantity & Duration				Rate (EGP/month)	Total (EGP)
Developers	3	x	4	months	1 0 , 0 0 0	1 2 0 , 0 0 0
Tester	1	x	2	months	8 , 0 0 0	1 6 , 0 0 0
Designer	1	x	5	months	9 , 0 0 0	4 5 , 0 0 0

Infrastructure Cost

Item	Cost (EGP)
Server & Hosting	2 0 , 0 0 0
	1 0 0 0 0

Total Project Cost = 2 1 1 , 0 0 0 EGP

Maintenance Cost

Item	Cost (EGP/year)
Annual Maintenance & Support	2 0 , 0 0 0

Proposed Selling Price

The proposed selling price to the university = 9 0 , 0 0 0 EGP per year

